Kingdom of Saudi Arabia

Ministry of Higher Education

Umm Al-Qura University

College of Computers in Al-Lith

Computer Science Department

Smart Agriculture Monitoring System

Course Name: Software Engineering

Instructor:

1. Introduction

The Smart Agriculture Monitoring System is a web-based application developed to support farm management and agricultural monitoring activities. The system provides an organized platform that allows users to manage farms, fields, .sensors, crops, and irrigation schedules through an easy-to-use interface

The project was developed as part of the Software Engineering course and follows the Software Development Life Cycle (SDLC) including requirements analysis, system design, implementation, testing, and deployment. The system aims to improve the management of agricultural resources and provide accurate. monitoring capabilities for farm operations

2. Requirements Analysis

2.1. Functional Requirements

The system provides the following functional requirements:

- Add, update, and delete farms
- Add, update, and delete fields
- Add and manage sensors
- Add and manage crops
- Create and manage irrigation schedules
- Display farm information and statistics
- Monitor sensor readings
- Organize agricultural data efficiently

2.2. Non-Functional Requirements

The system satisfies the following non-functional requirements:

- User-friendly interface
- Fast system response
- Reliable operation
- Easy maintenance and future expansion
- Data validation and input verification
- Clear and organized presentation of information

3. System Design

The design phase was completed using UML diagrams to represent the structure and behavior of the system. These diagrams helped in understanding system. relationships and interactions before implementation

3.1. Class Diagram

The Class Diagram illustrates the main entities within the system and the relationships between them. The design includes Farm, Field, Sensor, Sensor .Reading, Crop, and Irrigation Schedule classes

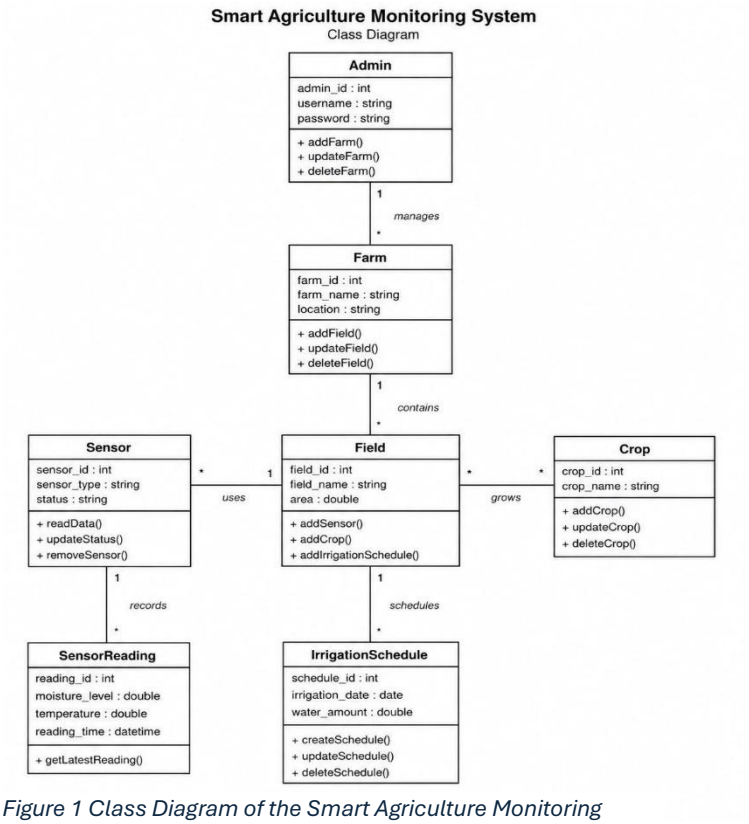

Figure 1 Class Diagram of the Smart Agriculture Monitoring

3.2. Activity Diagram

The Activity Diagram describes the workflow of the system and shows how users . interact with the application to perform agricultural management tasks

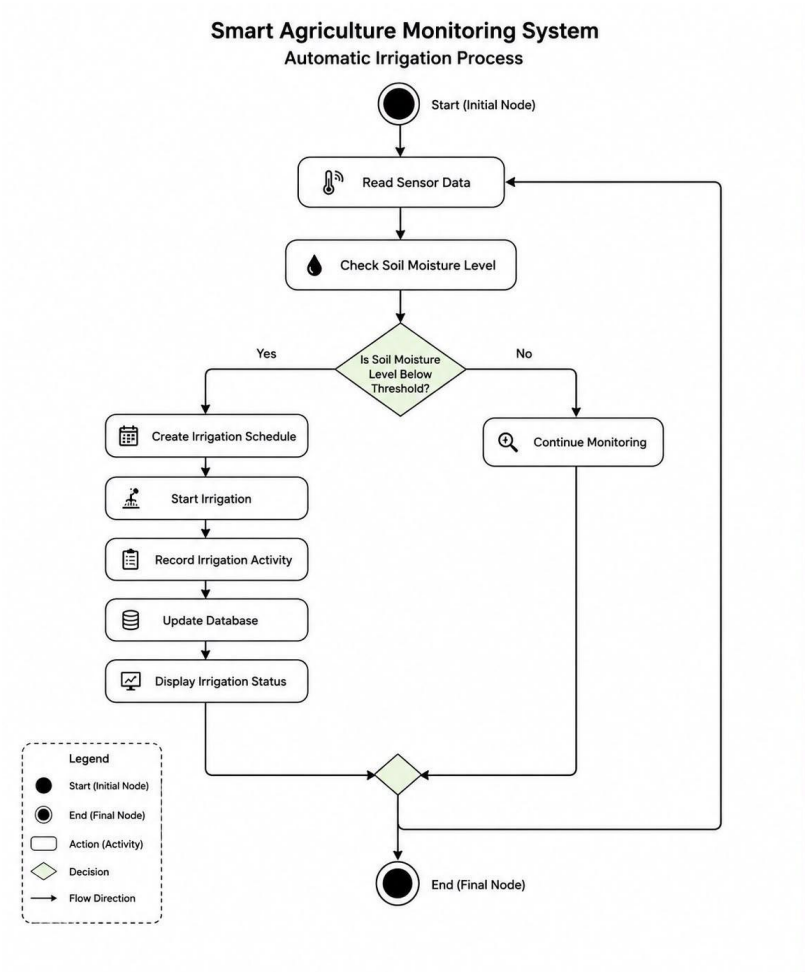

Figure 2 Activity Diagram of the Smart Agriculture Monitoring System

3.3. Sequence Diagram

The Sequence Diagram demonstrates the communication between system components when performing operations such as managing farms, fields, .sensors, and irrigation schedules

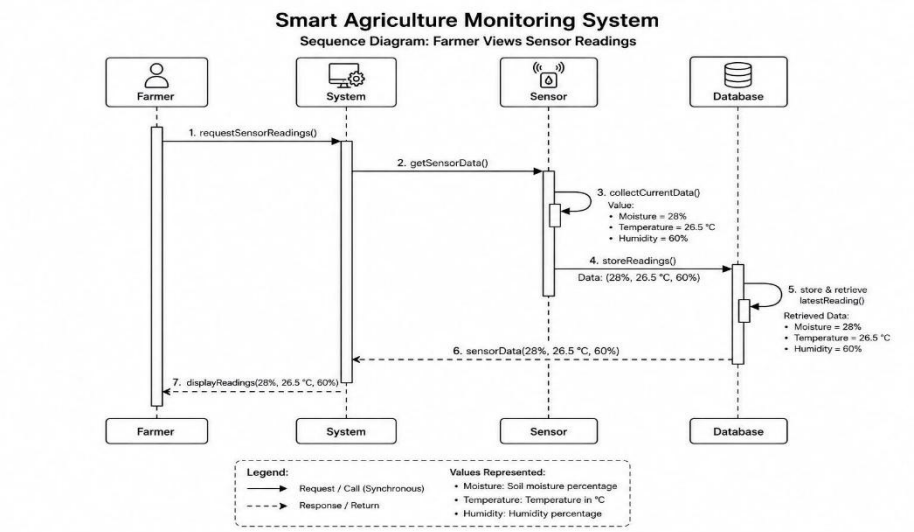

Figure 3 : Sequence Diagram of the Smart Agriculture Monitoring System

4. Implementation

The Smart Agriculture Monitoring System was implemented as a web application using HTML, CSS, and JavaScript. The implementation phase focused on transforming the system design into a functional application that satisfies all project requirements.

The application includes several modules that allow users to manage farms, fields, sensors, crops, and irrigation schedules. The dashboard provides a .centralized view of agricultural information and system statistics

- The implementation includes:
- Farm Management Module
- Field Management Module
- Sensor Management Module
- Crop Management Module
- Irrigation Schedule Management Module
- Dashboard Statistics Module
  Data Validation Mechanisms

Figure 6 System Dashboard

Figure 5 Farm Management Module

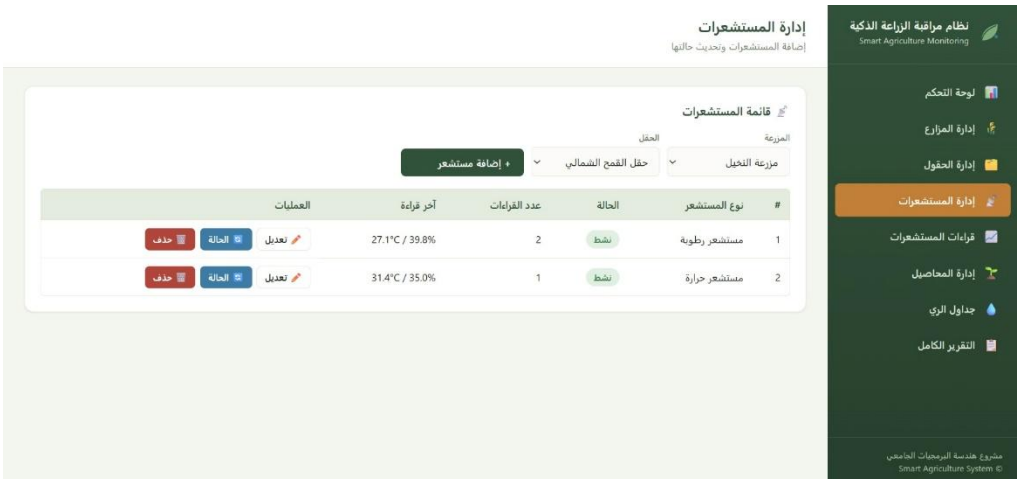

Figure 4 Sensor Management Module

5. Testing

Testing was performed to ensure that all system functionalities operate correctly and meet the project requirements. Different testing techniques were applied to . verify system quality and reliability

5.1. Unit Testing

Unit Testing was conducted to verify the correctness of individual functions and modules. Each function was tested independently to ensure that it produces the expected results.

The testing focused on:

- Farm operations
- Field operations
- Sensor operations
- Crop operations
- Irrigation schedule operations

Test ID	Function	Test Data	Expected Result	Actual Result	Status
UT-01	Add Farm	Green Valley Farm	Expected Result	Farm added successfully	Pass
UT-02	Edit Farm	New Farm Name	Farm updated successfully	Farm updated successfully	Pass
UT-03	Delete Farm	Existing Farm	Farm deleted successfully	Farm deleted successfully	Pass
UT-04	Add Field	Wheat North Field	Field added successfully	Field added successfully	Pass
UT-05	Add sensor	Temperature Sensor	Sensor added successfully	Sensor added successfully	Pass
UT-06	Add Crop	Wheat Crop	Crop added successfully	Crop added successfully	Pass
UT-07	Create Schedule	Daily 08:00 AM	Crop added successfully	Schedule created successfully	Pass
UT-08	View Farm Information	Existing Farm	Farm information displayed	Farm information displayed	Pass

Table 1: Unit Testing Results

The results indicate that all tested functions performed successfully and met their expected outcomes

5.2. Integration Testing

Integration Testing was performed to verify that different system modules work together correctly. The interaction between farms, fields, sensors, crops, and irrigation schedules was tested.

Test ID	Scenario	Expected Result	Actual Result	Status
IT-01	Add Farm → Add Field	Field linked to farm	Success	Pass
IT-02	Add Field → Add Sensor	Sensor linked to field	Success	Pass
IT-03	Add Field → Add Crop	Crop linked to field	Success	Pass
IT-04	Add Field → Create Schedule	Schedule linked to field	Success	Pass
IT-05	Add Sensor → Add Reading	Reading stored in sensor	Success	Pass
IT-06	Display Complete Farm Information	Reading stored in sensor	Success	Pass

Table 2 Integration Testing Results

The results confirm that the integrated components communicate properly and exchange data successfully.

5.3. Static Testing

Static Testing was carried out through code inspection and review without executing the system. The purpose was to identify potential issues related to . code structure, comments, naming conventions, and software design

Test ID	Check Item	Expected Result	Actual Result	Status
ST-01	Code readability	Code is readable	Passed	Pass
ST-02	Comments availability	Comments exist	Passed	Pass
ST-03	Naming conventions	Proper names used	Passed	Pass
ST-04	Syntax validation	No syntax errors	Passed	Pass
ST-05	Structure review	OOP structure followed	Passed	Pass

Table 3: Static Testing Results

The review confirmed that the code follows appropriate programming standards .and maintains a clear structure

5.4. Dynamic Testing

Dynamic Testing was conducted by running the system and validating its behavior during execution. Various user interactions and system operations were . tested to ensure proper functionality

Test ID	Action	Expected Result	Actual Result	Status
DT-01	Open system	System loads successfully	Success	Pass
DT-02	Add farm through interface	Farm added	Success	Pass
DT-03	Add field through interface	Field added	Success	Pass
DT-04	Add sensor through interface	Sensor added	Success	Pass
DT-05	Add crop through interface	Crop added	Success	Pass
DT-06	Create irrigation schedule	Schedule created	Success	Pass
DT-07	View dashboard statistics	Statistics displayed	Success	Pass
DT-08	Navigate between pages	Pages load correctly	Success	Pass

Table 4: Dynamic Testing Results

The system successfully completed all dynamic test cases without major issues

6. Deployment

The Smart Agriculture Monitoring System was deployed using GitHub as a version control and project hosting platform. GitHub was used to manage .project files, maintain version history, and organize development activities

The repository contains:

- Source code
- Project documentation
- Testing artifacts
- Supporting project files

GitHub Repository:
https://github.com/gta877267-ui/-Smart-Agriculture-Monitoring-System.git

7. Meeting Logs

Meeting logs were maintained throughout the project development process to . document discussions, decisions, progress updates, and task assignments

The complete meeting records are provided separately in the Meeting_logs.xlsx file.

8. Conclusion

The Smart Agriculture Monitoring System successfully achieved its intended objectives by providing a practical and organized solution for agricultural monitoring and farm management. The system allows users to manage farms, fields, sensors, crops, and irrigation schedules through an interactive and user. friendly interface

Throughout the project, software engineering principles were applied, including requirements analysis, UML-based design, implementation, testing, and deployment. The testing results demonstrated that the system functions. correctly and satisfies the specified requirements

The project provided valuable experience in software development, system design, testing methodologies, and project management, making it a successful Software Engineering project.